

Nível de Microarquitetura (parte 5)



UNIFEI

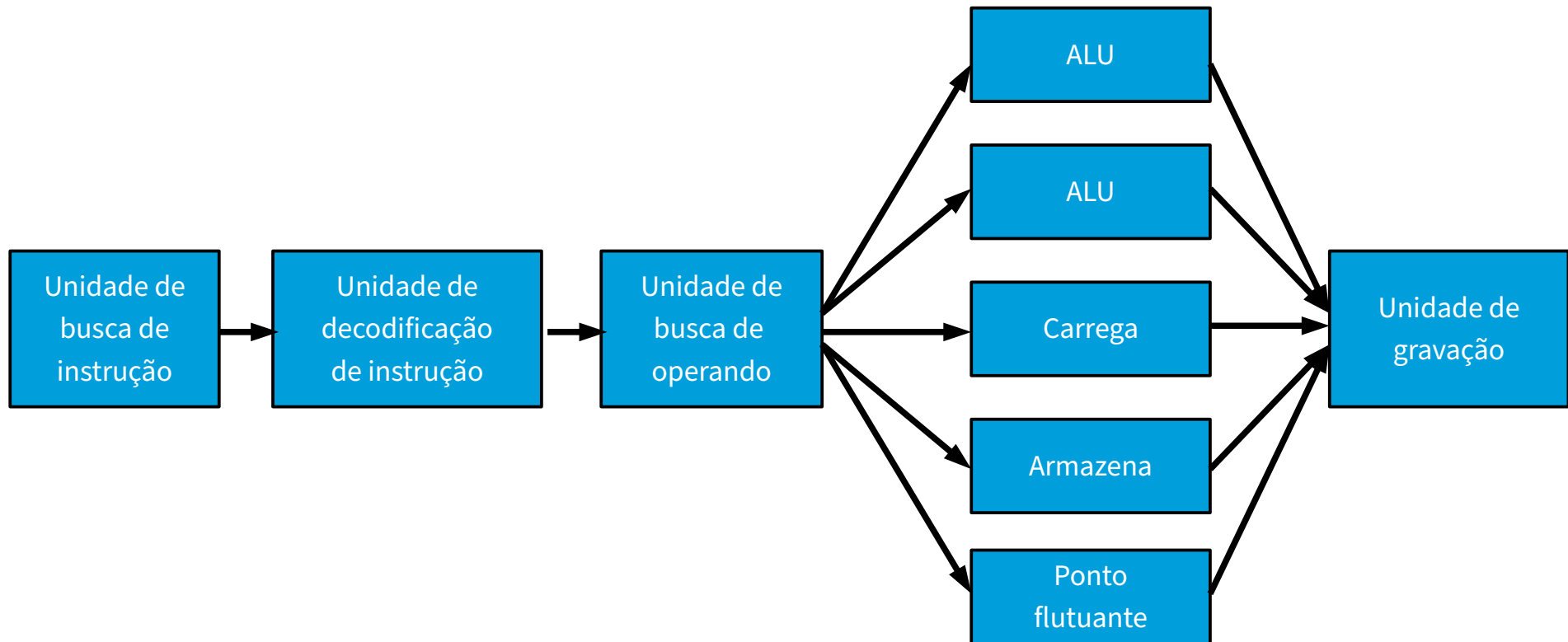
Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

Execução fora de ordem e renomeação de registrador

Grande parte das CPUs modernas tem paralelismo e também são **superescalares**, como mostra a figura:



Em geral, isso significa que há uma unidade de busca que retira palavras de instrução da memória antes que elas sejam necessárias, para alimentar uma unidade de decodificação. A unidade de decodificação emite as instruções decodificadas para as unidades funcionais adequadas para execução. Em alguns casos ela pode desmembrar instruções individuais em microoperações antes de emití-las para as unidades funcionais, dependendo do que as unidades funcionais podem fazer.

Claro que o projeto da máquina é mais simples se as instruções forem executadas na ordem em que são buscadas (considerando, por enquanto, que o algoritmo de previsão de desvio nunca faça uma previsão errada). Contudo, a execução em ordem nem sempre resulta em desempenho ótimo devido às dependências entre instruções. Se uma instrução precisar de um valor calculado pela instrução anterior, a segunda não pode começar a executar até que a primeira tenha produzido o valor necessário. Nessa situação (uma dependência RAW – Read After Write), a segunda instrução tem de esperar. Também existem outros tipos de dependência, como veremos em breve.

Em uma tentativa de contornar esses problemas e produzir melhor desempenho, algumas CPUs permitem saltar instruções dependentes para chegar a instruções futuras que não são dependentes. Não é preciso dizer que o algoritmo de escalonamento de instruções internas usado deve causar o mesmo efeito que causaria se o programa fosse executado na ordem escrita. Agora demonstraremos como a reordenação de instruções funciona usando um exemplo detalhado.

Nosso exemplo de máquina tem oito registradores visíveis para o programador, R0 até R7. Todas as instruções aritméticas usam três registradores: dois para os operandos e um para o resultado, igual à Mic-4. Vamos considerar que, se uma instrução for decodificada no ciclo n , a execução inicia no ciclo $n + 1$. Para uma instrução simples, como uma adição ou subtração, a escrita retroativa no registrador de destino ocorre ao final do ciclo $n + 2$. Para uma instrução mais complicada, como uma multiplicação, a escrita retroativa ocorre ao final do ciclo $n + 3$. Para tomar o exemplo realista, permitiremos que a unidade de decodificação emita até duas instruções por ciclo de relógio. Há várias CPUs escalares comerciais que podem emitir quatro ou até seis instruções por ciclo de relógio.

A primeira coluna dá o número do ciclo e a segunda dá o número da instrução. A terceira coluna relaciona a instrução decodificada. A quarta informa qual instrução está sendo emitida com um máximo de duas por ciclo de relógio. A quinta informa qual instrução foi retirada, ou concluída. Lembre-se de que nesse exemplo estamos exigindo emissão em ordem, bem como conclusão em ordem, portanto a instrução $k + 1$ não pode ser emitida até que a instrução k tenha sido emitida e a instrução $k + 1$ não pode ser retirada, ou seja, não pode ser escrita retroativamente no registrador de destino até que a instrução k tenha sido retirada. As outras 16 colunas são discutidas logo adiante.

					Registadores lidos								Registadores escritos							
Ci	#	Decod.	Emit.	Ret.	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=R0* R1	1		1	1										1				
	2	R4=R0+R2	2		2	1	1									1	1			

Após decodificar a instrução, a unidade de decodificação tem de decidir se pode ou não emití-la imediatamente. Para tomar essa decisão, a unidade de decodificação precisa conhecer o estado de todos os registradores. Se, por exemplo, a instrução corrente precisar de um registrador cujo valor ainda não foi calculado, ela não pode ser emitida e a CPU deve protelar.

A tabela de pontuação também tem contadores para monitorar os registradores usados como destino. Uma vez que só é permitida uma escrita por vez, esses contadores podem ter um bit de largura. As 16 colunas da extrema direita na tabela mostram a tabela de pontuação.

Para ver se uma instrução pode ser emitida, são aplicadas as seguintes regras:

1. Se qualquer operando estiver sendo escrito, não emita (dependência RAW).
2. Se o registrador de resultado estiver sendo lido, não emita (dependência WAR).
3. Se o registrador de resultado estiver sendo escrito, não emita (dependência WAW).

Execução em ordem

					Registradores lidos								Registradores escritos							
Ci	#	Decod.	Emit.	Ret.	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=RO* R1	1		1	1										1				
	2	R4=RO+R2	2		2	1	1									1	1			
2	3	R5=RO+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	-		3	2	1									1	1	1		
3					3	2	1									1	1	1		
4				1	2	1	1										1	1		
				2	1	1												1		
				3																
5			4			1			1										1	
	5	R7=R1*R2	5			2	1		1										1	1
6	6	R1=RO-R2	-			2	1		1										1	1
7				4		1	1													1
8				5																

CPU superescalar com emissão em ordem e conclusão em ordem.

Ci	#	Decod.	Emit.	Ret.	Registradores lidos								Registradores escritos							
					0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
9					1		1							1						
	7	$R3 = R3 * R1$			1		1							1						
10					1		1							1						
11				6																
12			7			1		1								1				
	8	$R1 = R4 + R4$	–			1		1								1				
13						1		1								1				
14						1		1								1				
15				7																
16			8						2					1						
17									2					1						
18				8																

CPU superescalar com emissão em ordem e conclusão em ordem.

Execução fora de ordem e renomeação de registradores

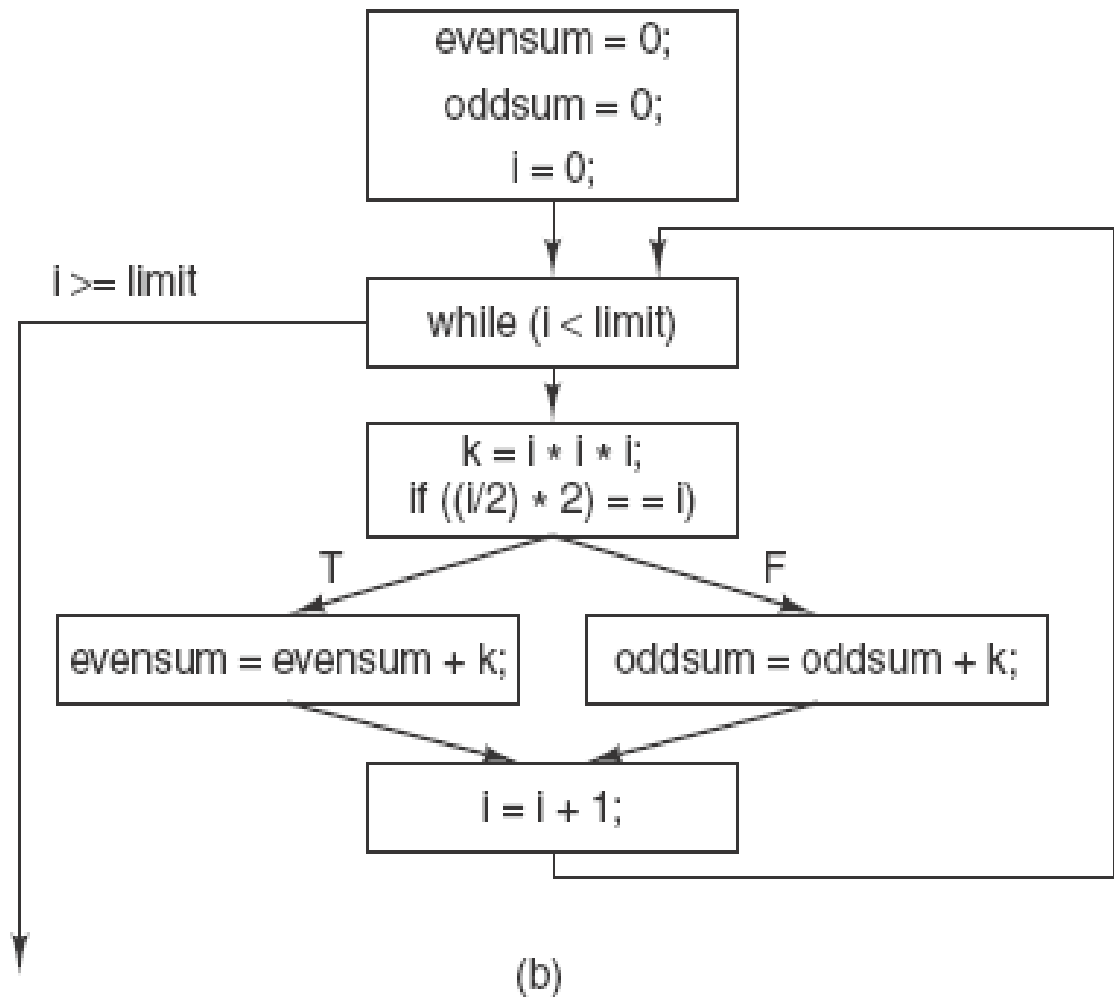
Ci	#	Decod.	Emit.	Ret.	Registradores lidos								Registradores escritos							
					0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=RO* R1	1		1	1										1				
	2	R4=RO+R2	2		2	1	1									1	1			
2	3	R5=RO+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	—		3	2	1									1	1	1		
3	5	R7=R1*R2	5		3	3	3									1	1	1		1
	6	S1=RO-R2	6		4	3	3									1	1	1		1
				2	3	3	2									1		1		1
4			4		3	4	2		1							1	1	1	1	1
	7	R3=R3*S1	—		3	4	2		1							1		1	1	1
	8	S2=R4+R4	8		3	4	2		3							1		1	1	1
				1	2	3	2		3									1	1	1
				3	1	2	2		3										1	1
5				6		2	1		3					1					1	1
6			7			2	1	1	3					1		1			1	1
				4		1	1	1	2					1		1				1
				5				1	2					1		1				
				8				1								1				
7								1								1				
8								1								1				
9				7																

Operação de uma CPU superescalar com emissão de instrução fora de ordem e conclusão de instrução fora de ordem.

Execução especulativa

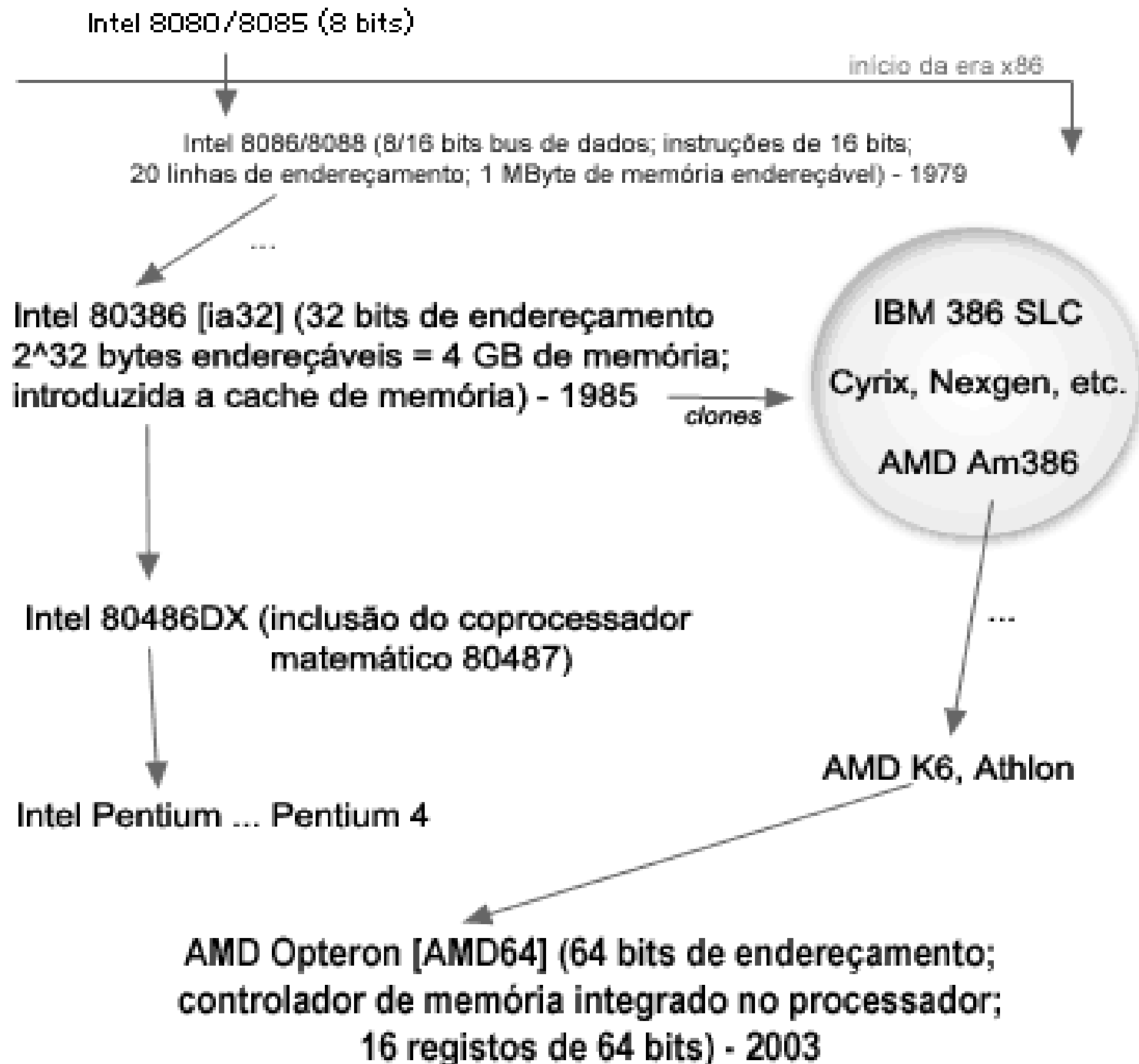
```
evensum = 0;
oddsum = 0;
i = 0;
while (i < limit) {
    k = i * i * i;
    if (((i/2) * 2) == i)
        evensum = evensum + k;
    else
        oddsum = oddsum + k;
    i = i + 1;
}
```

(a)



(b)

- (a) **Fragmento de um programa.**
(b) **Gráfico de blocos básicos correspondente.**

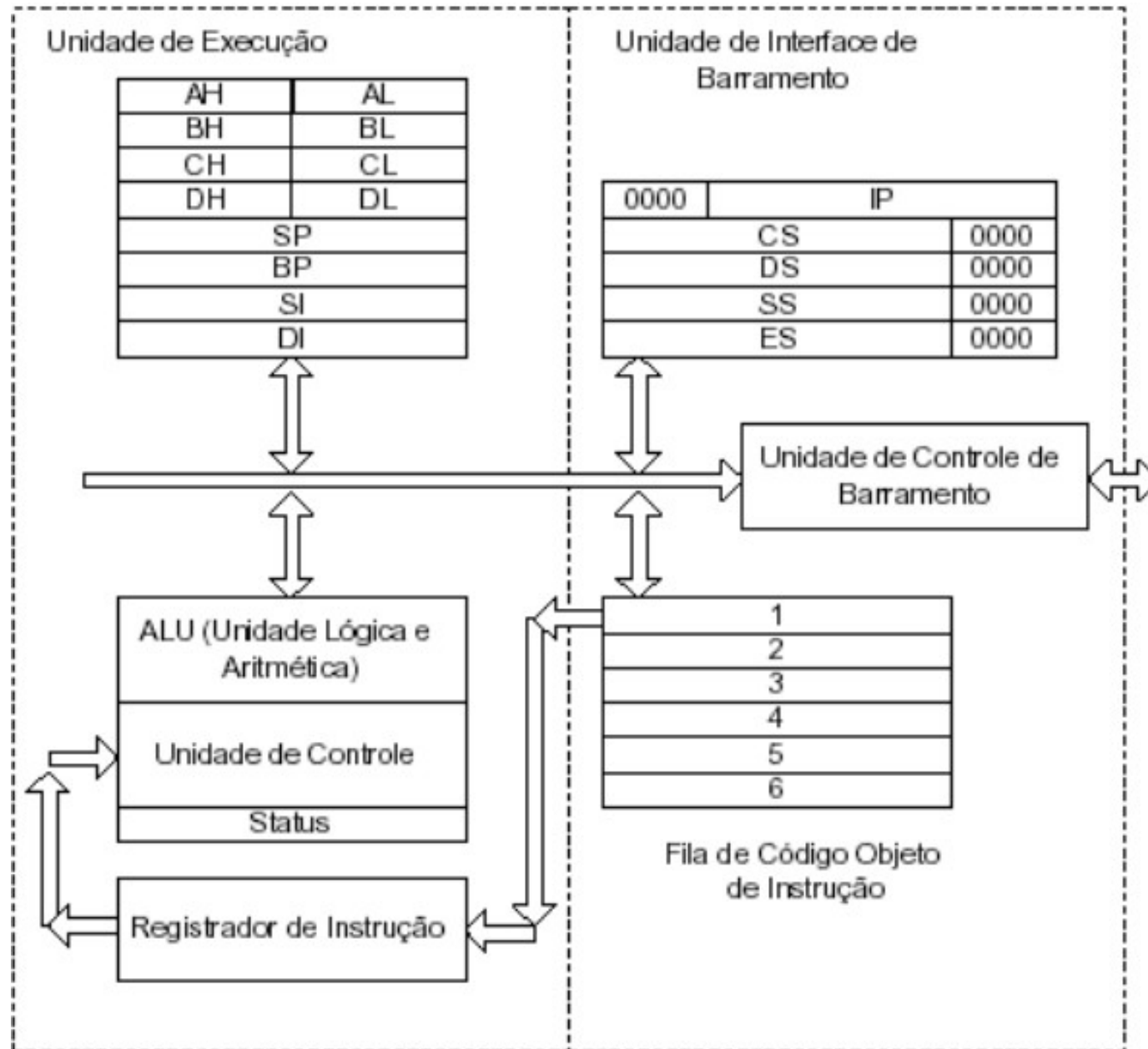


Pentium 4

- Parece um processador CISC
- Conjunto de instruções imenso e desajeitado
- Operações com inteiros de 8, 16 e 32 bits
- Operações de ponto flutuante de 32 e 64 bits
- Somente 8 registradores visíveis, não há 2 deles que sejam exatamente iguais
- Comprimento das instruções de 1 a 17 bytes

**UM DESASTRE DE UMA
ARQUITETURA HERDADA**

Arquitetura dos processadores 8086/88



Arquitetura dos processadores 8086/88

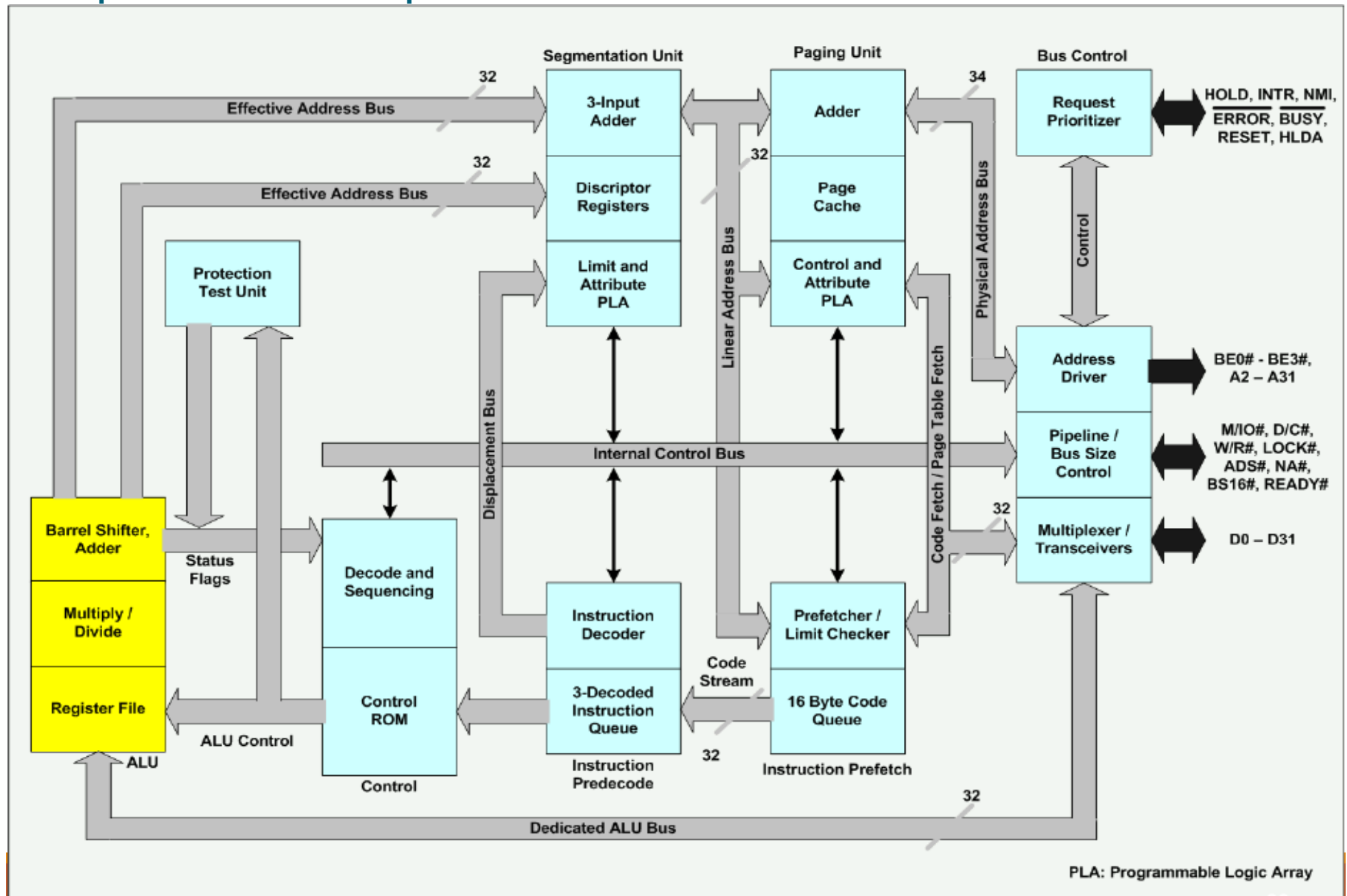
Registadores

				0	1	
AX	AH	AL	Acumulador	CF - Carry Flag	CY	NC
BX	BH	BL	Base	PF - Parity Flag	PE	PO
CX	CH	CL	Contador	AF - Auxiliary Carry Flag	AC	NA
DX	DH	DL	Dados	ZF - Zero Flag	ZR	NZ
	SP		Ponteiro de Pilha	SF - Sign Flag	NG	PL
	BP		Ponteiro de Base	TF - Trap Flag		
	SI		Índice de Fonte	IF - Interrupt Flag	EI	DI
	DI		Índice de Destino	DF - Direction Flag	DN	UP
	CS		Segmento de Código	OF - Overflow Flag	OV	NV
	DS		Segmento de Dados			
	ES		Segmento Extra			
	SS		Segmento de Pilha			
	IP		Ponteiro de Instrução			
	FLAGS		Sinalizadores			

7	6	5	4	3	2	1	0
SF	ZF		AF		PF		CF
				OF	DF	IF	TF
15	14	13	12	11	10	9	8

FLAGS

Arquitetura dos processadores 80386



Arquitetura dos processadores 80386

Registadores

x86 Registers

80386

8086

<i>EAX</i>		<i>AH</i>	<i>AL</i>	<i>AX</i>	(accumulator)
<i>EBX</i>		<i>BH</i>	<i>BL</i>	<i>BX</i>	(base register)
<i>ECX</i>		<i>CH</i>	<i>CL</i>	<i>CX</i>	(count register)
<i>EDX</i>		<i>DH</i>	<i>DL</i>	<i>DX</i>	(data register)
<i>ESI</i>				<i>SI</i>	(source index)
<i>EDI</i>				<i>DI</i>	(destination index)
<i>ESP</i>				<i>SP</i>	(stack pointer)
<i>EBP</i>				<i>BP</i>	(base pointer)
				<i>CS</i>	(code segment)
				<i>DS</i>	(data segment)
				<i>SS</i>	(stack segment)
				<i>ES</i>	(extra segment)
<i>FS</i>					
<i>GS</i>					
<i>EIP</i>				<i>IP</i>	(instruction pointer)
	<i>EFL</i>			<i>FL</i>	(flag)

Arquitetura dos processadores 486

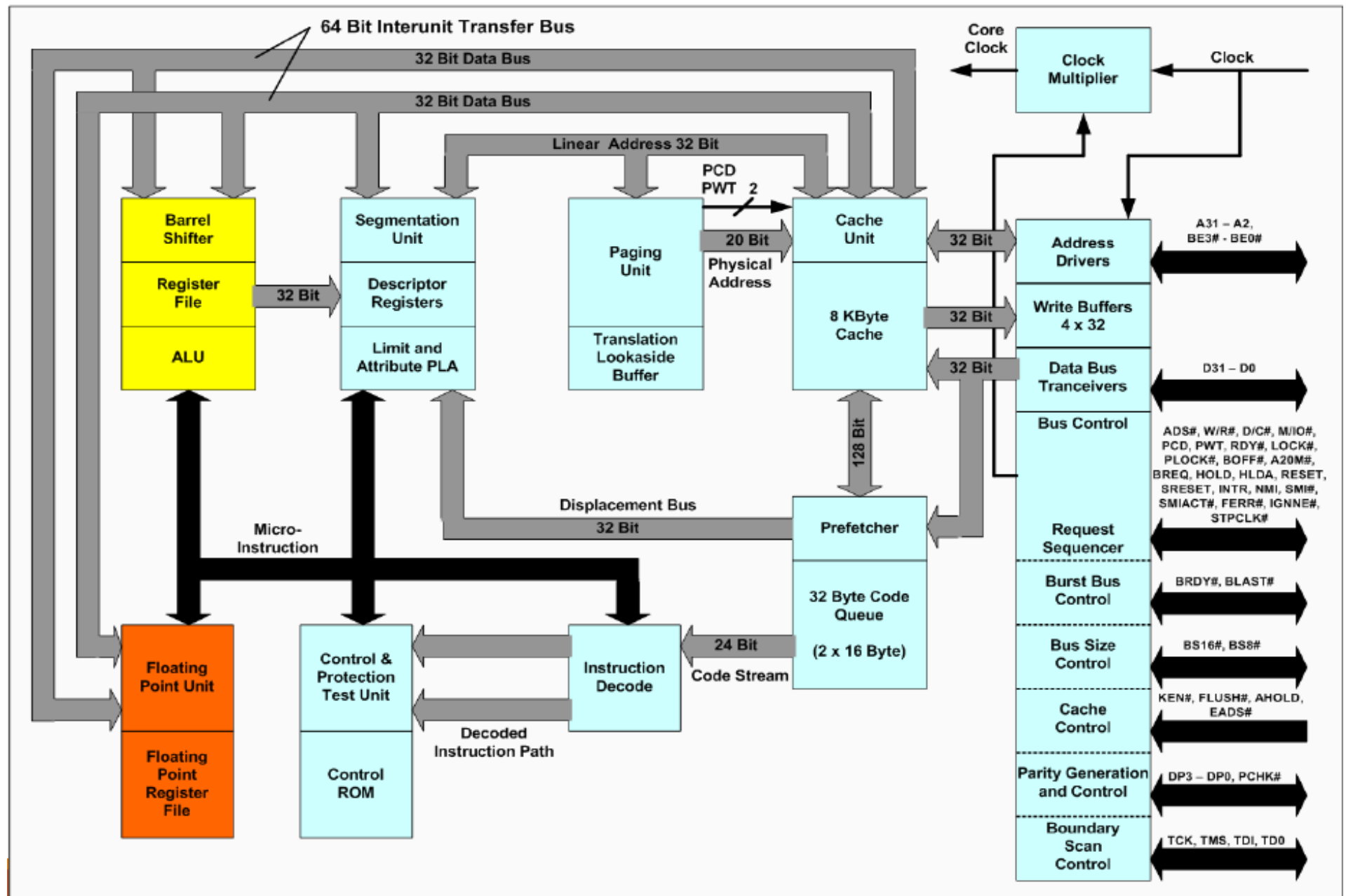
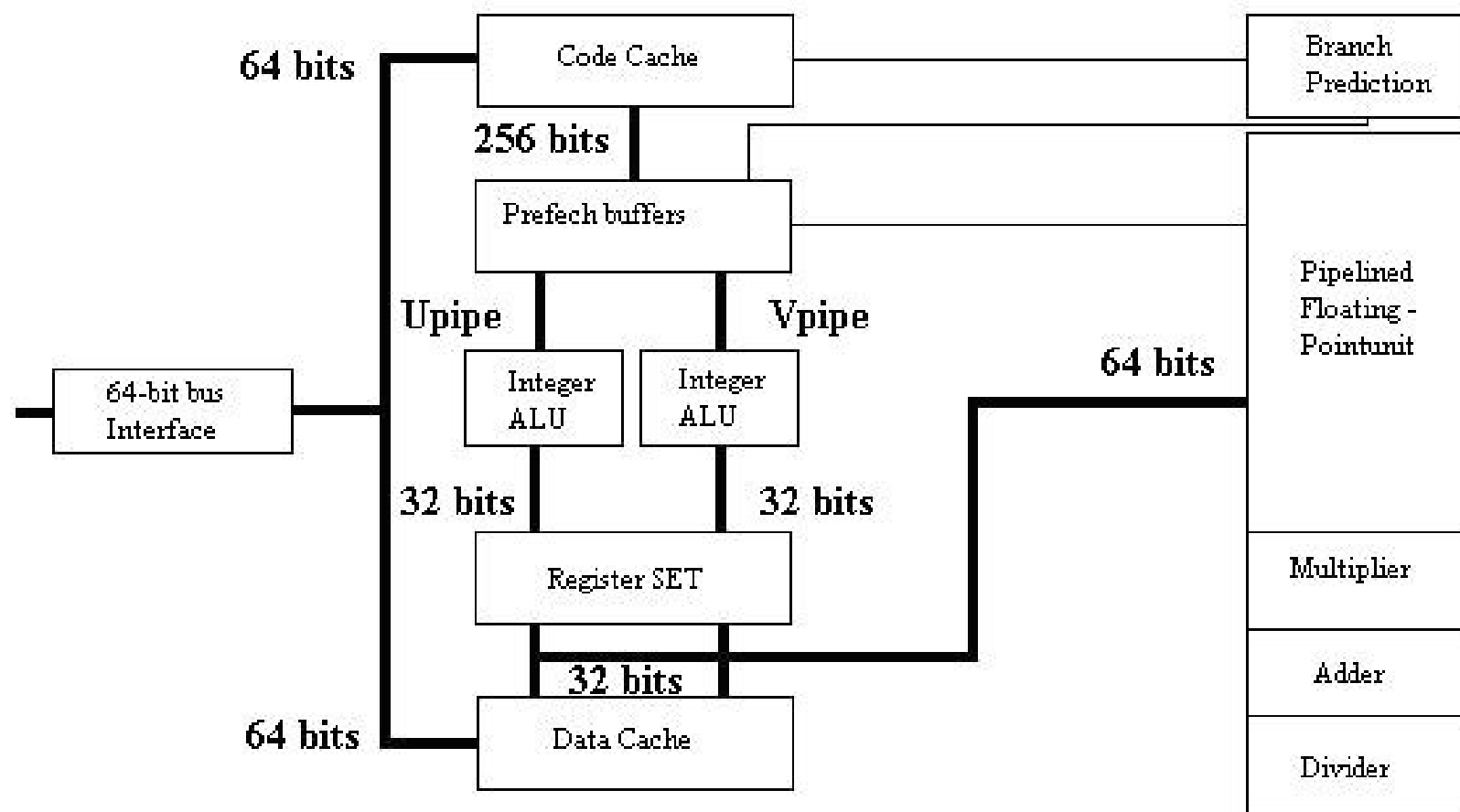
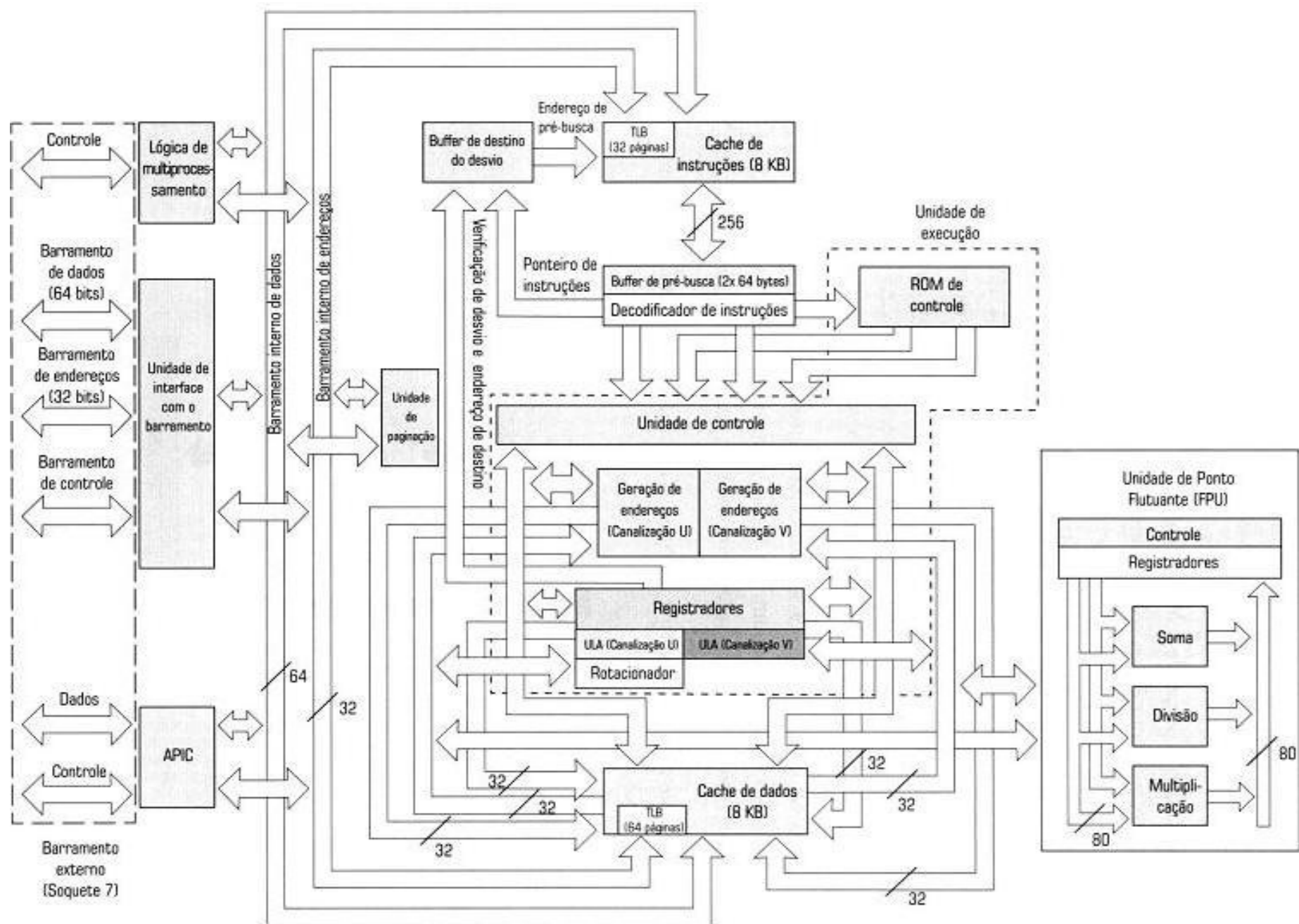


DIAGRAMA DE BLOCOS DO PROCESSADOR PENTIUM



Arquitetura dos processadores Pentium MMX



Arquitetura dos processadores Pentium Pro

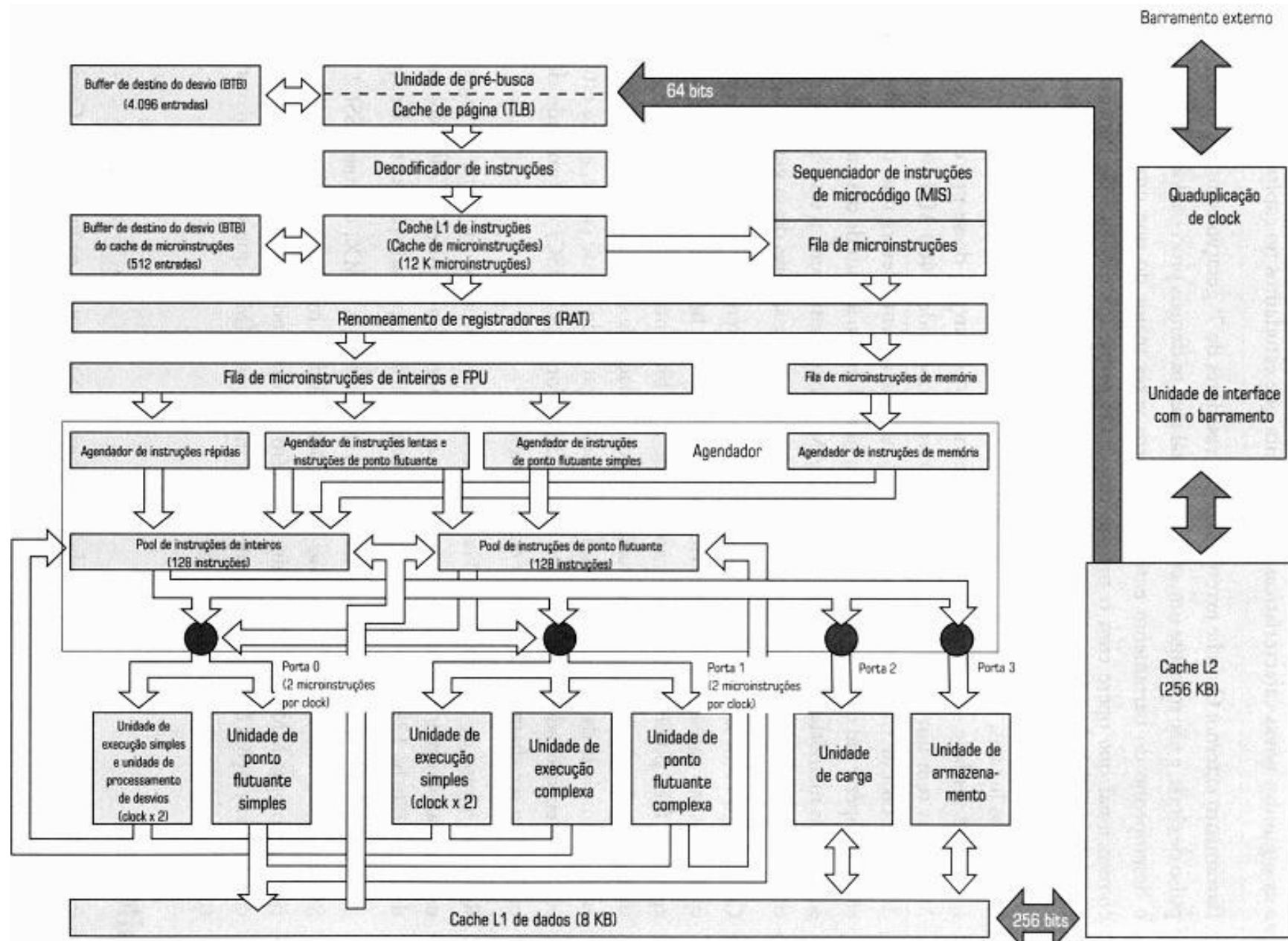


Diagrama em blocos do AMD-K6

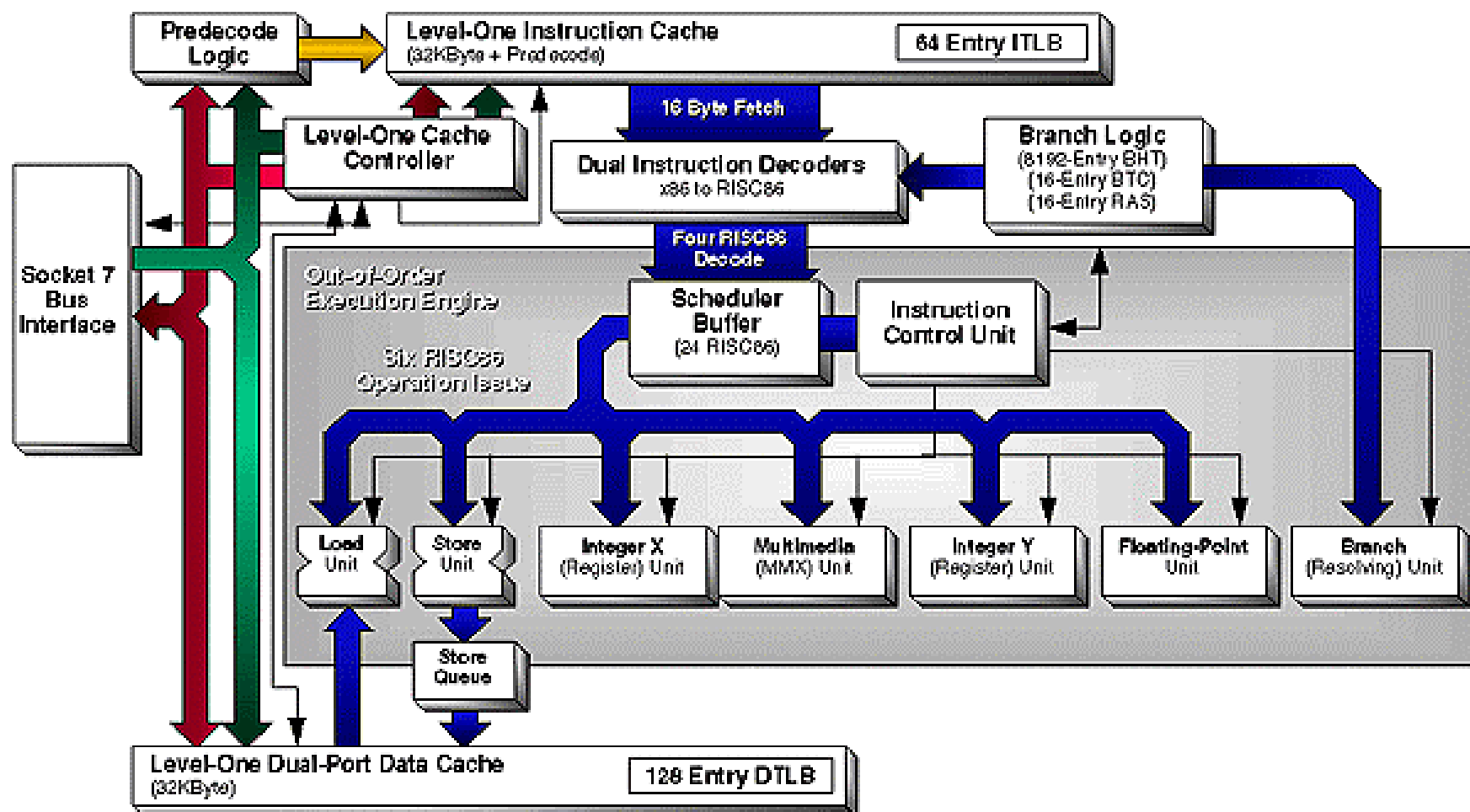
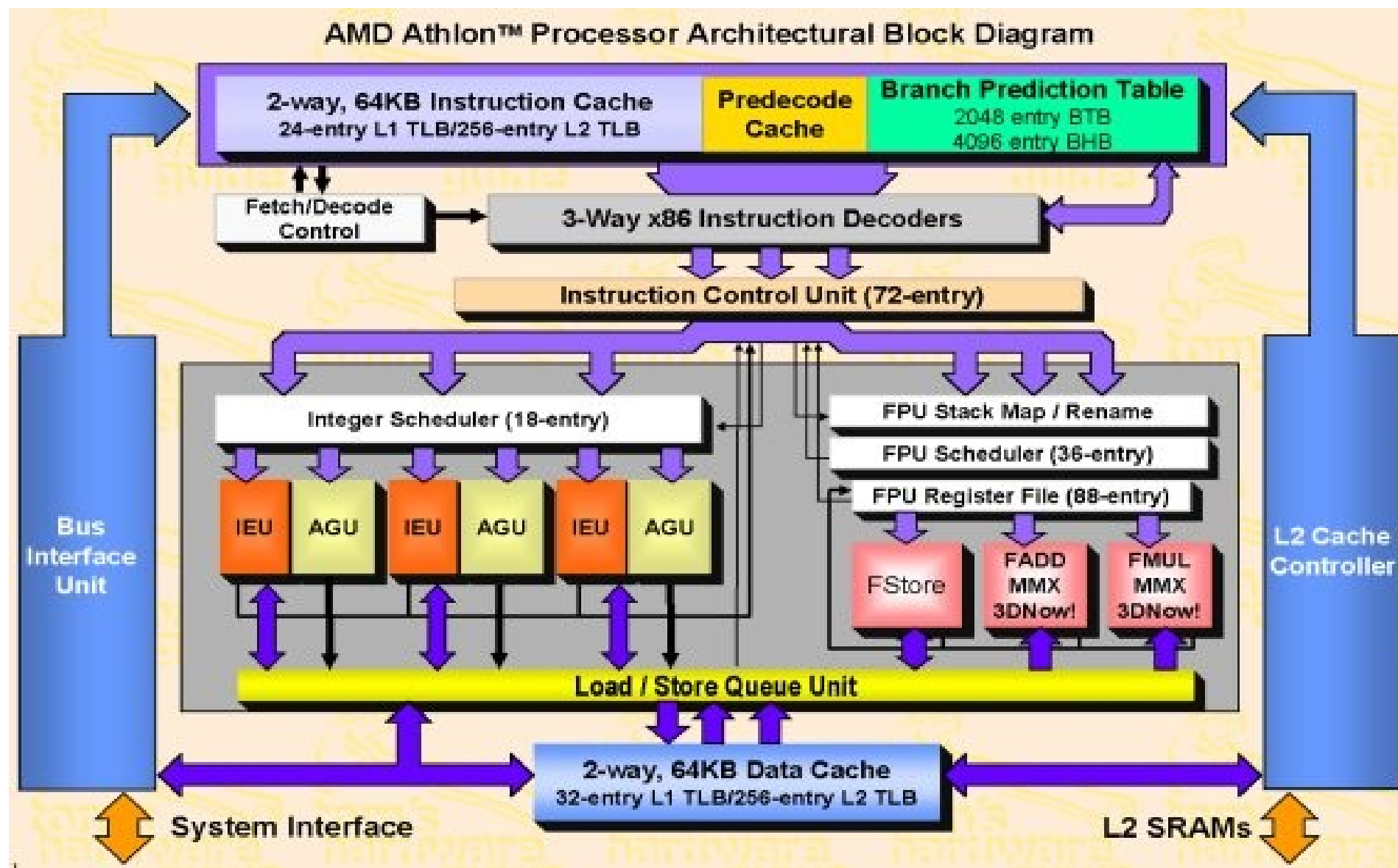
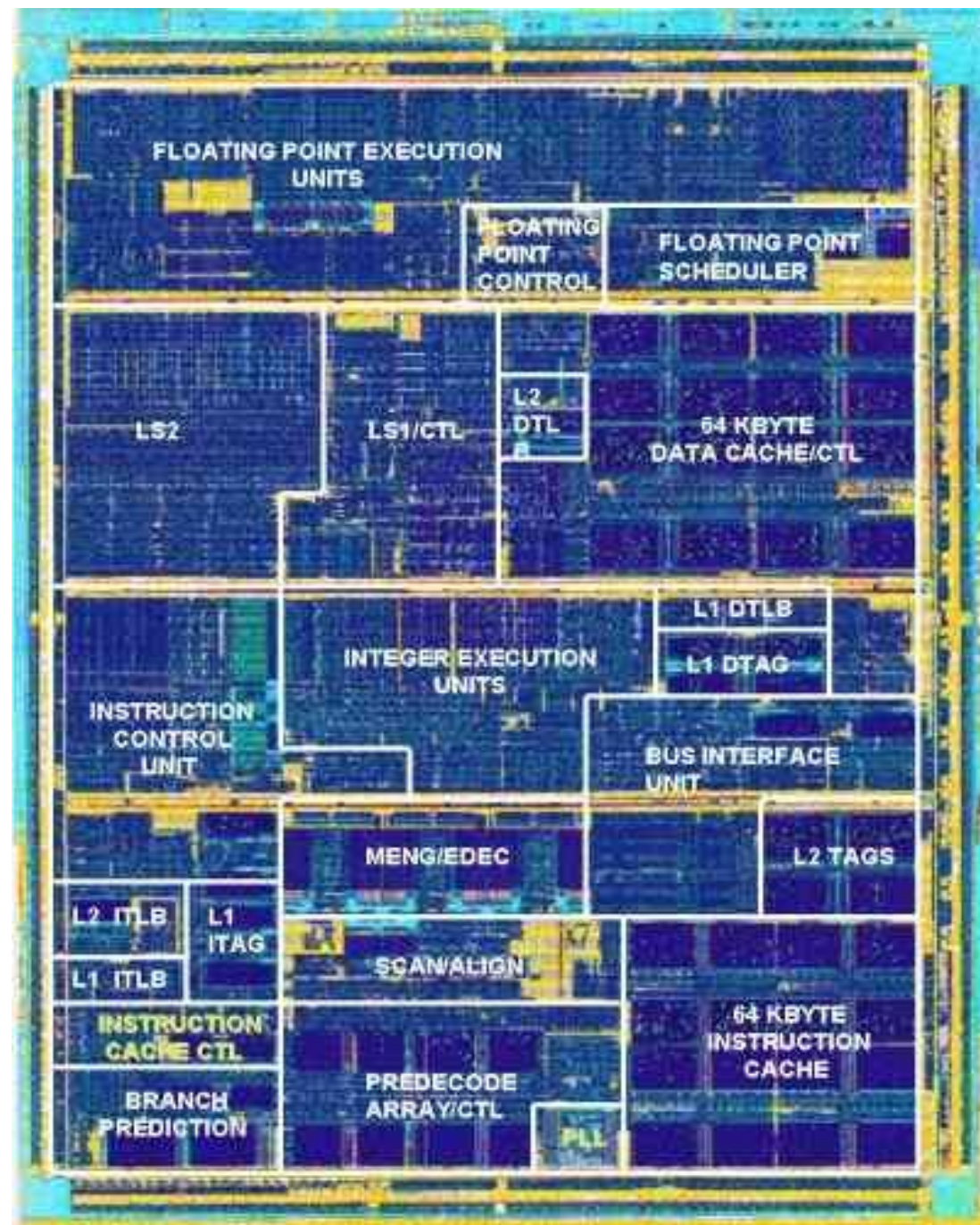


Diagrama em blocos da arquitetura do processador Athlon



Veja que a foto está dividida em pequenos retângulos, cada um mostrando a área ocupada por cada componente do processador.



Visão geral da microarquitetura NetBurst

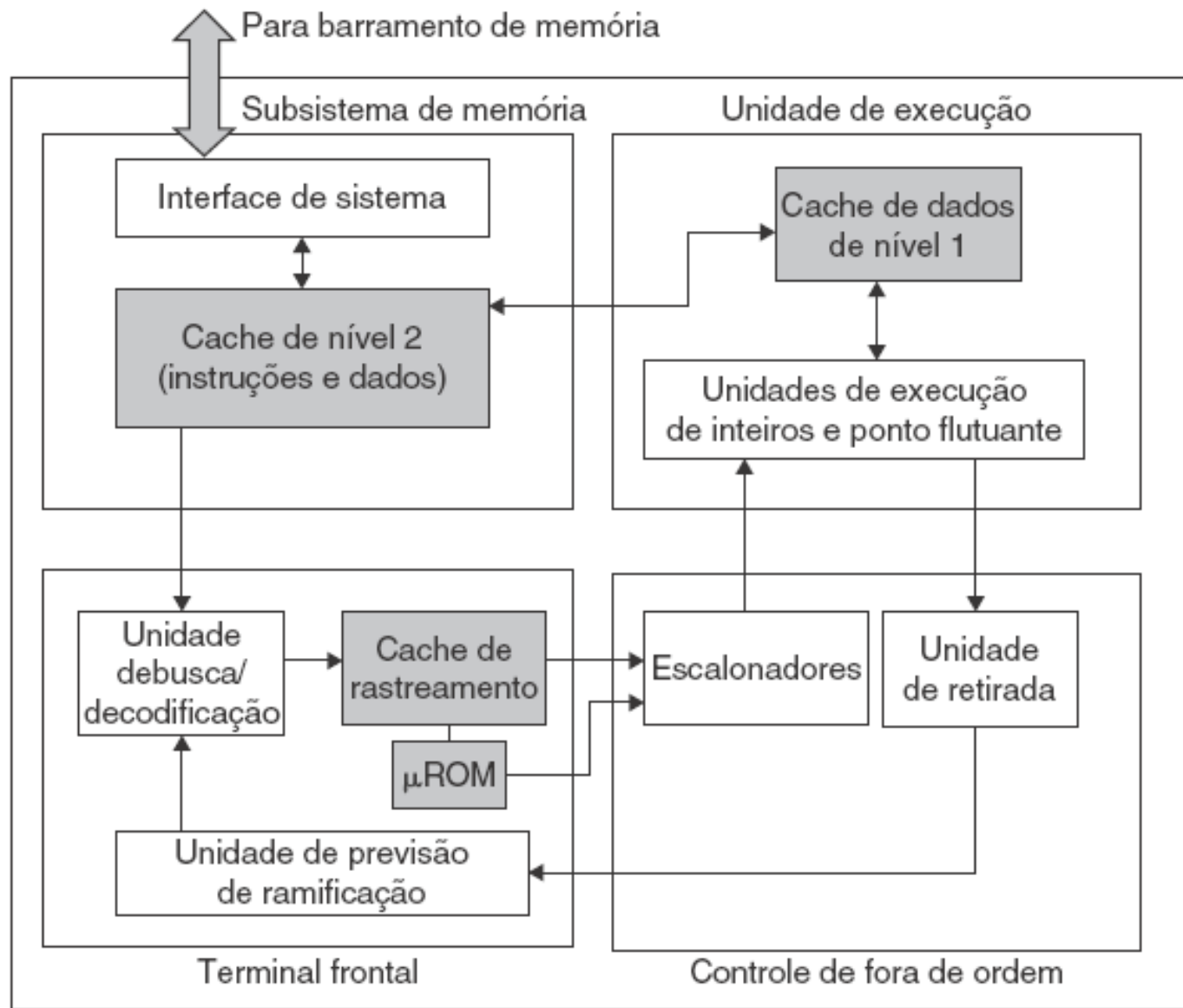
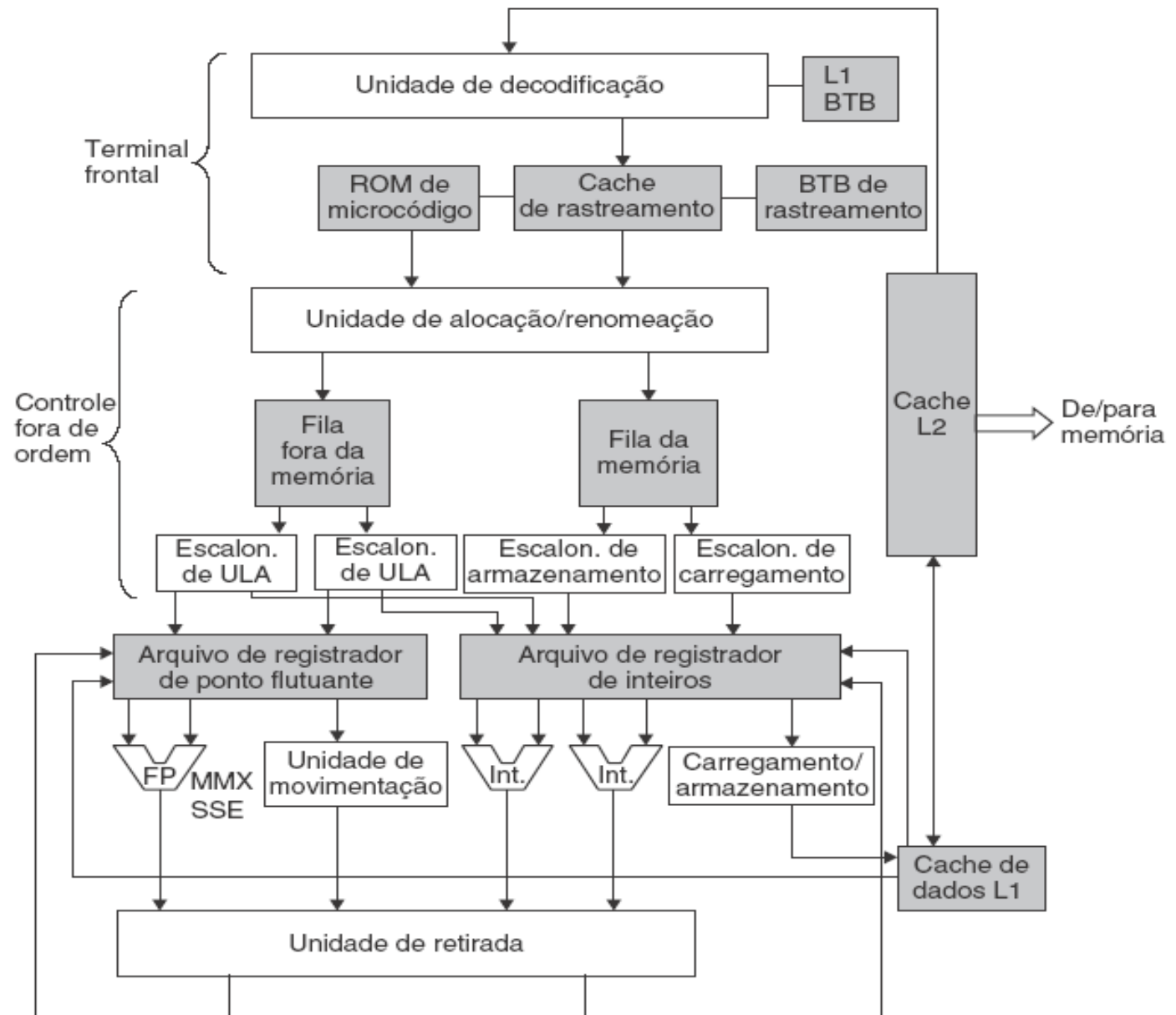


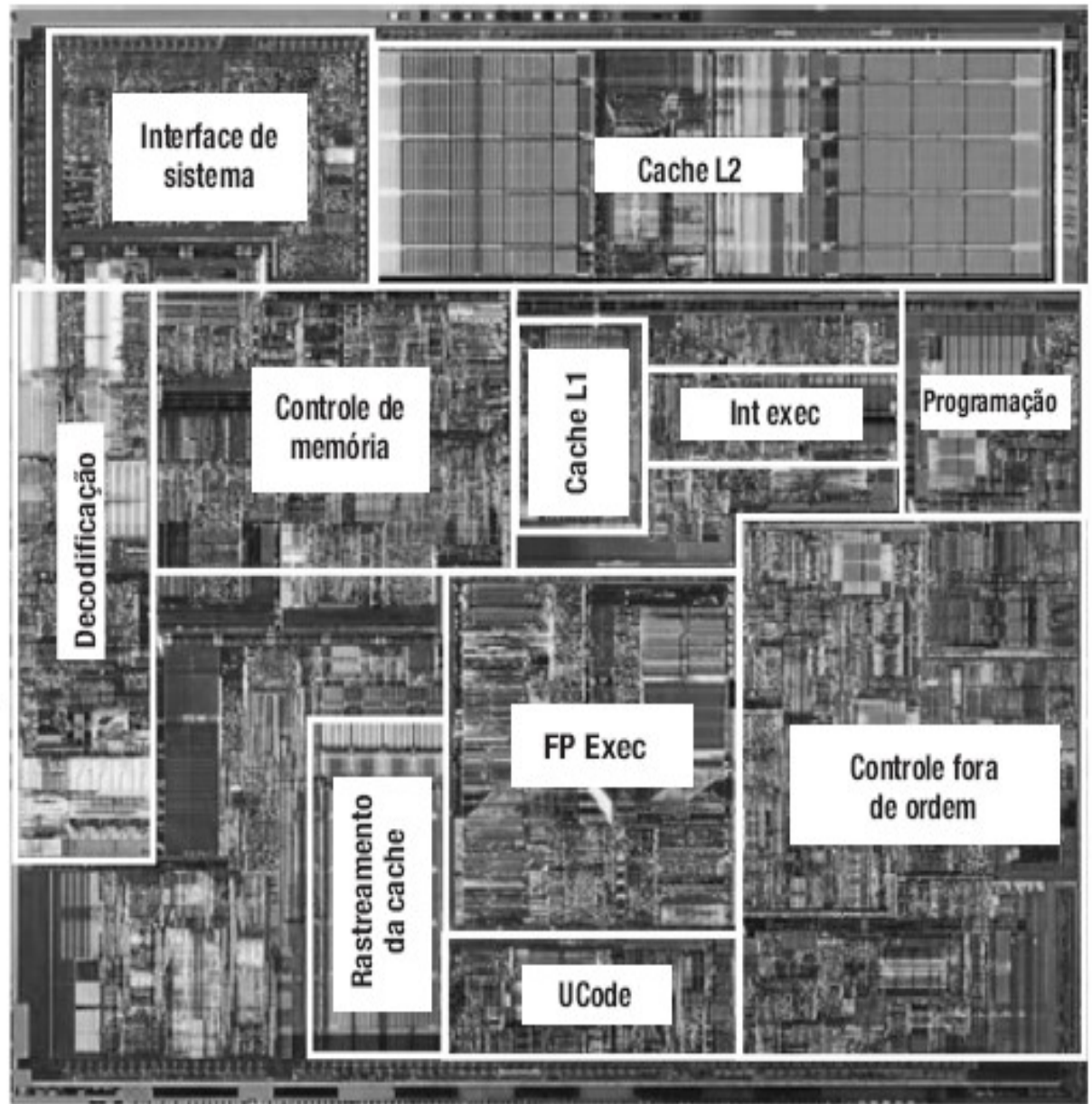
Diagrama de blocos do Pentium 4.

O paralelismo da NetBurst

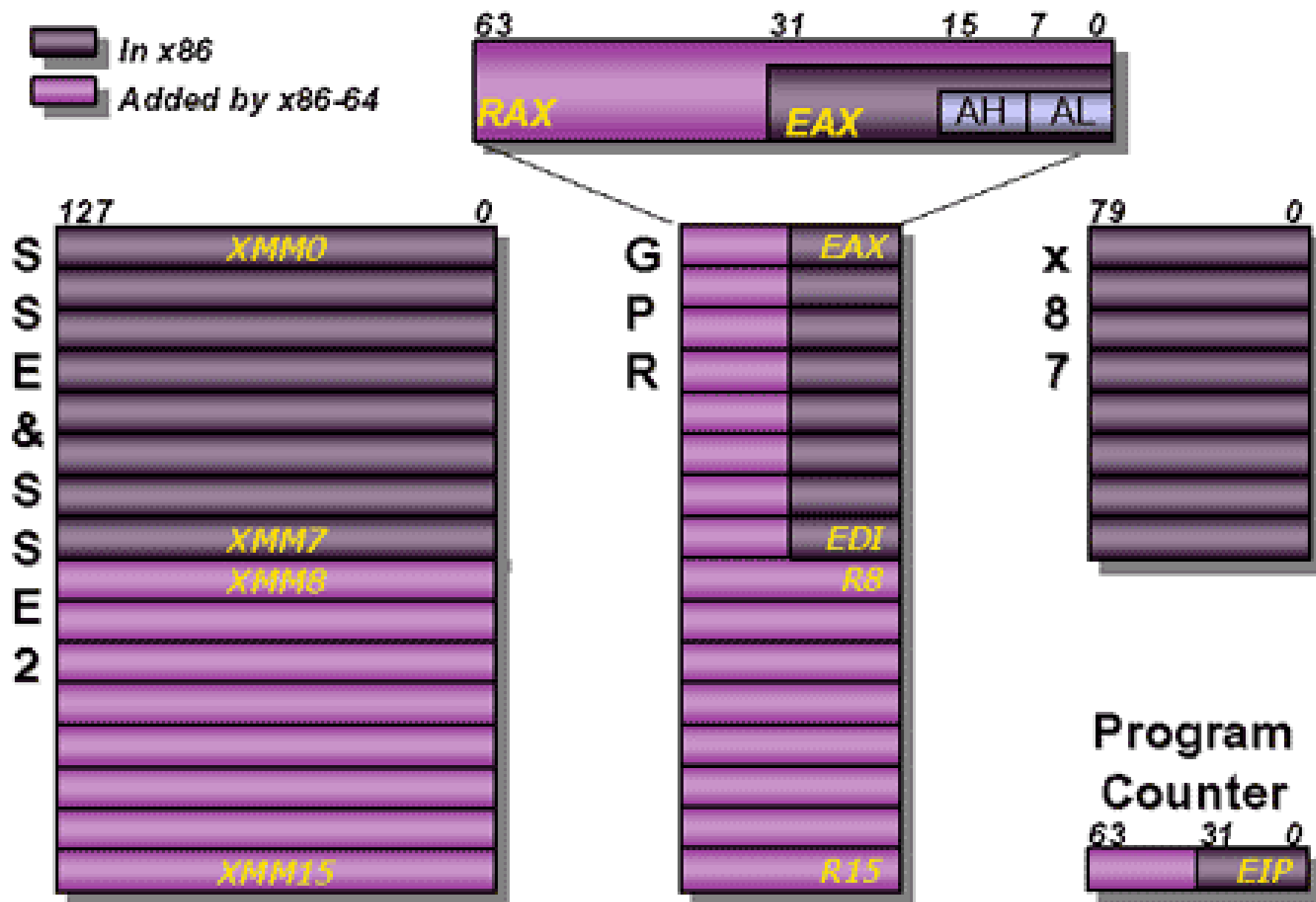
Visão
simplificada
do caminho
de dados do
Pentium 4.



Chip Pentium 4.
Direitos de
reprodução da
Intel
Corporation,
2003, utilização
permitida.

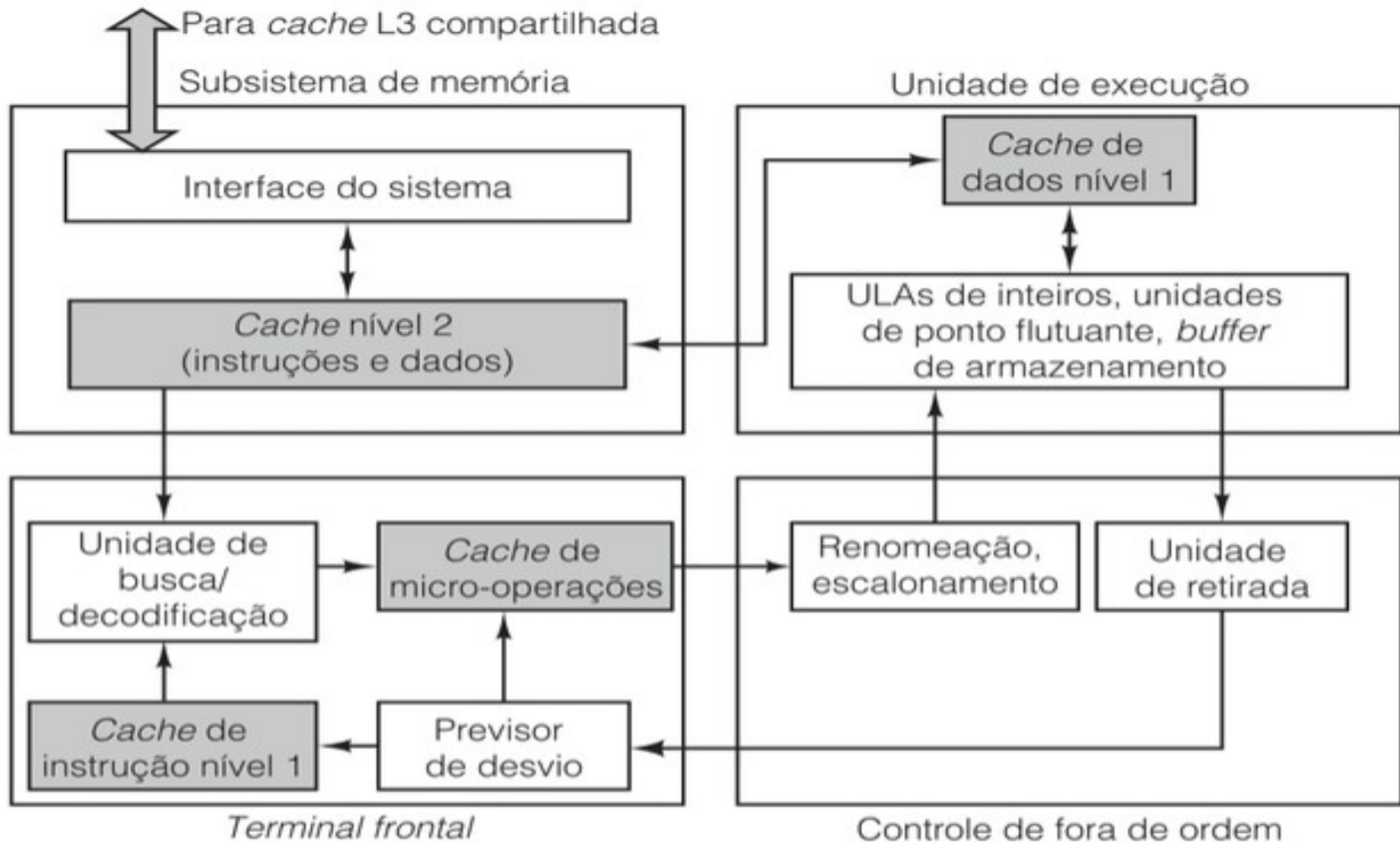


x86-64 Programmer's Model



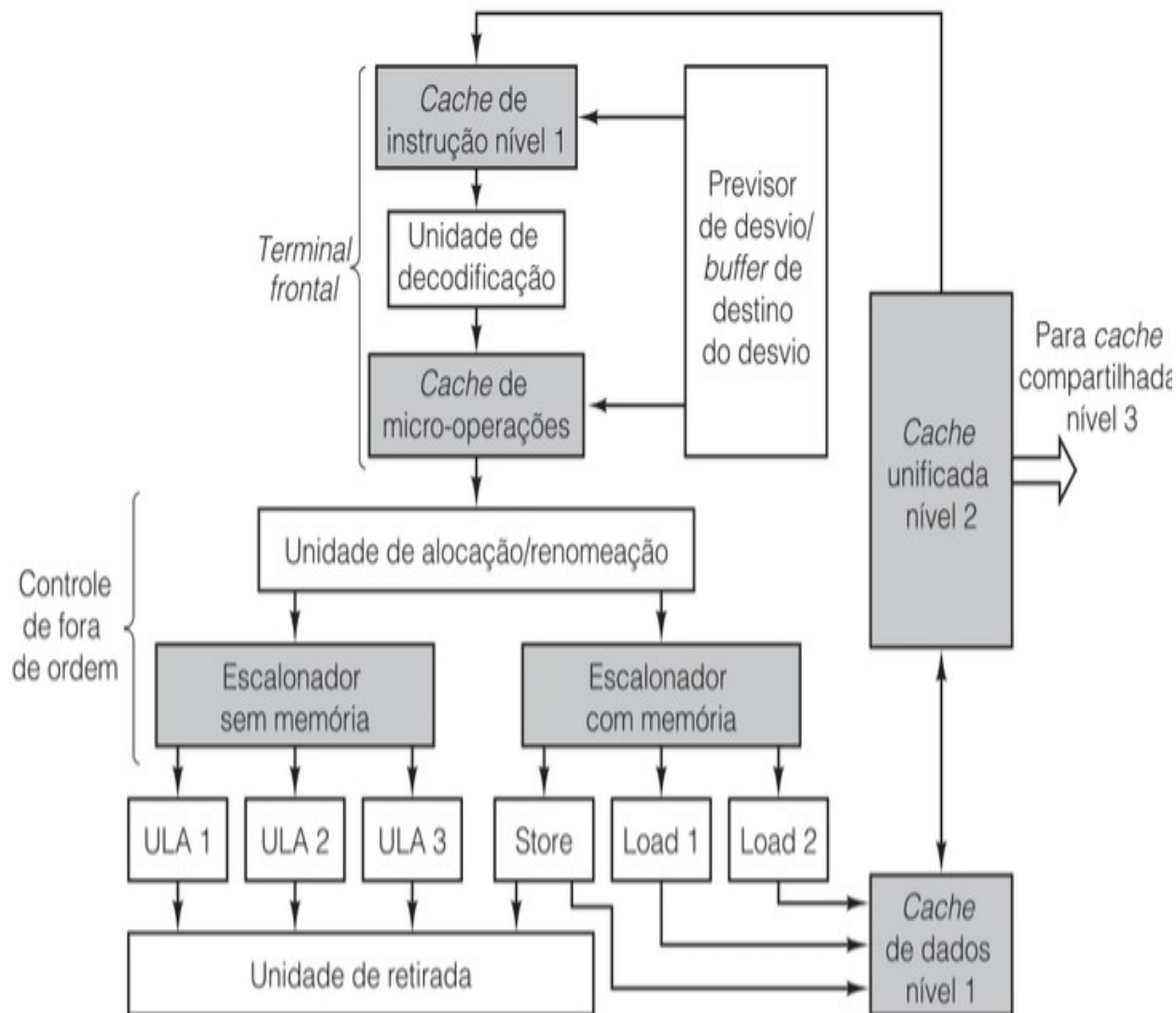
Core i7 – Sandy Bridge

Diagrama de blocos do Core i7.



Caminho de dados do Core i7

17 Visão simplificada do caminho de dados do Core i7.

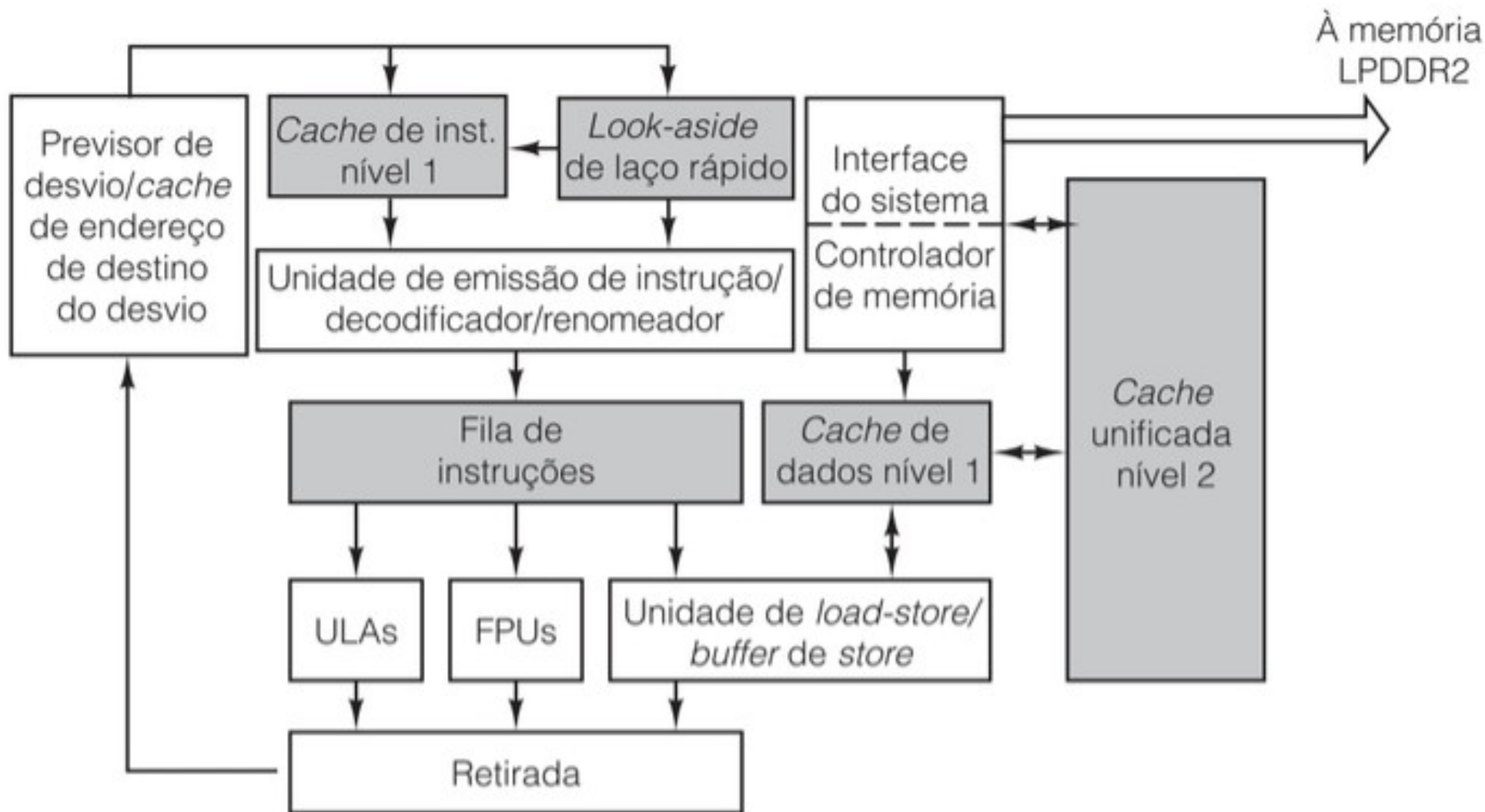


- Unidades funcionais

- ULA1 e a unidade de multiplicação de ponto flutuante
- ULA2 e a unidade de adição/subtração de ponto flutuante
- ULA3 e a unidade de processamento de desvio e comparação de ponto flutuante
- Instruções *store*
- Instruções *load1*
- Instruções *load2*

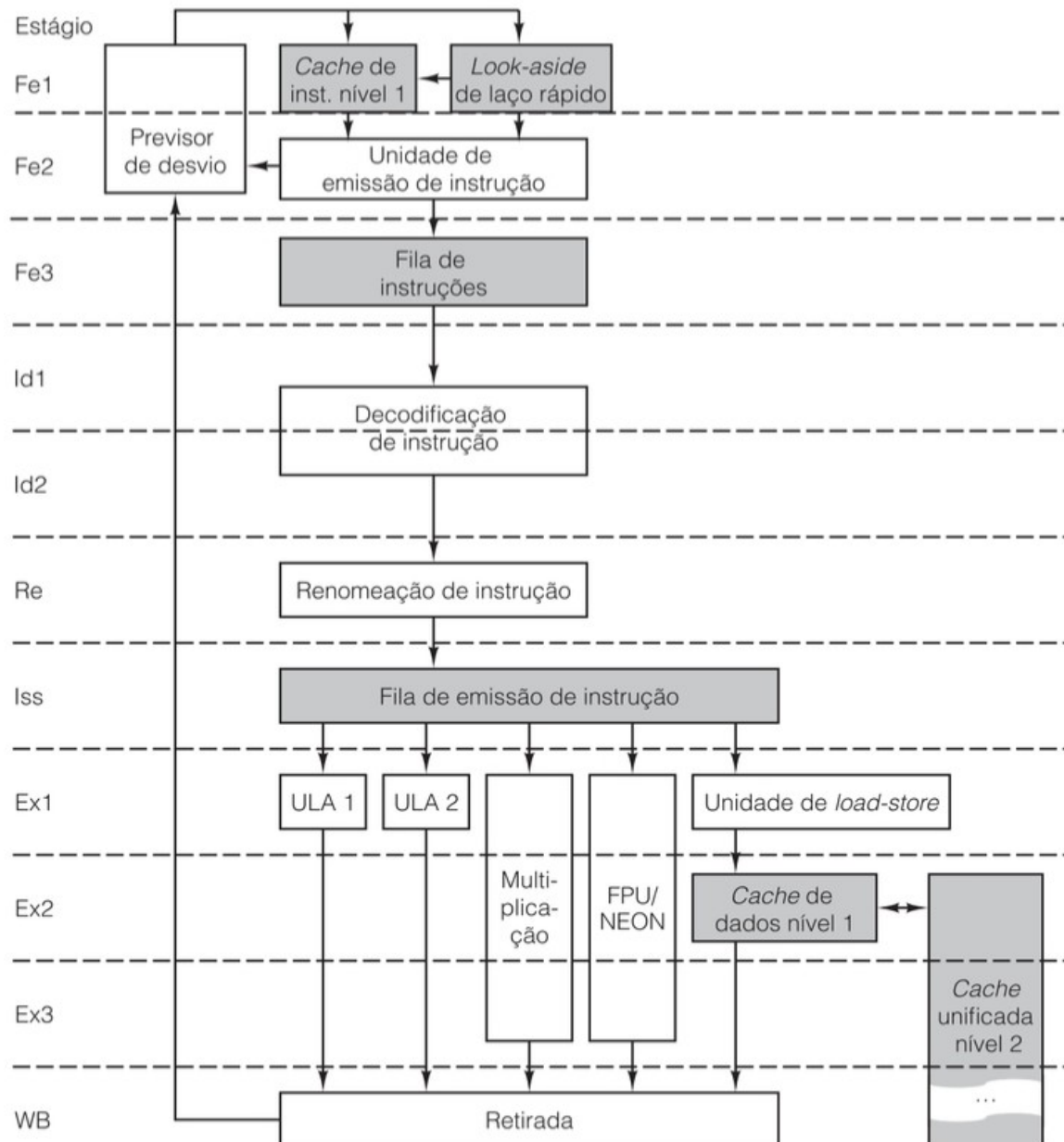
Cortex A9

Diagrama de blocos da microarquitetura Cortex A9 da CPU OMAP4430.



Pipeline do Cortex A9

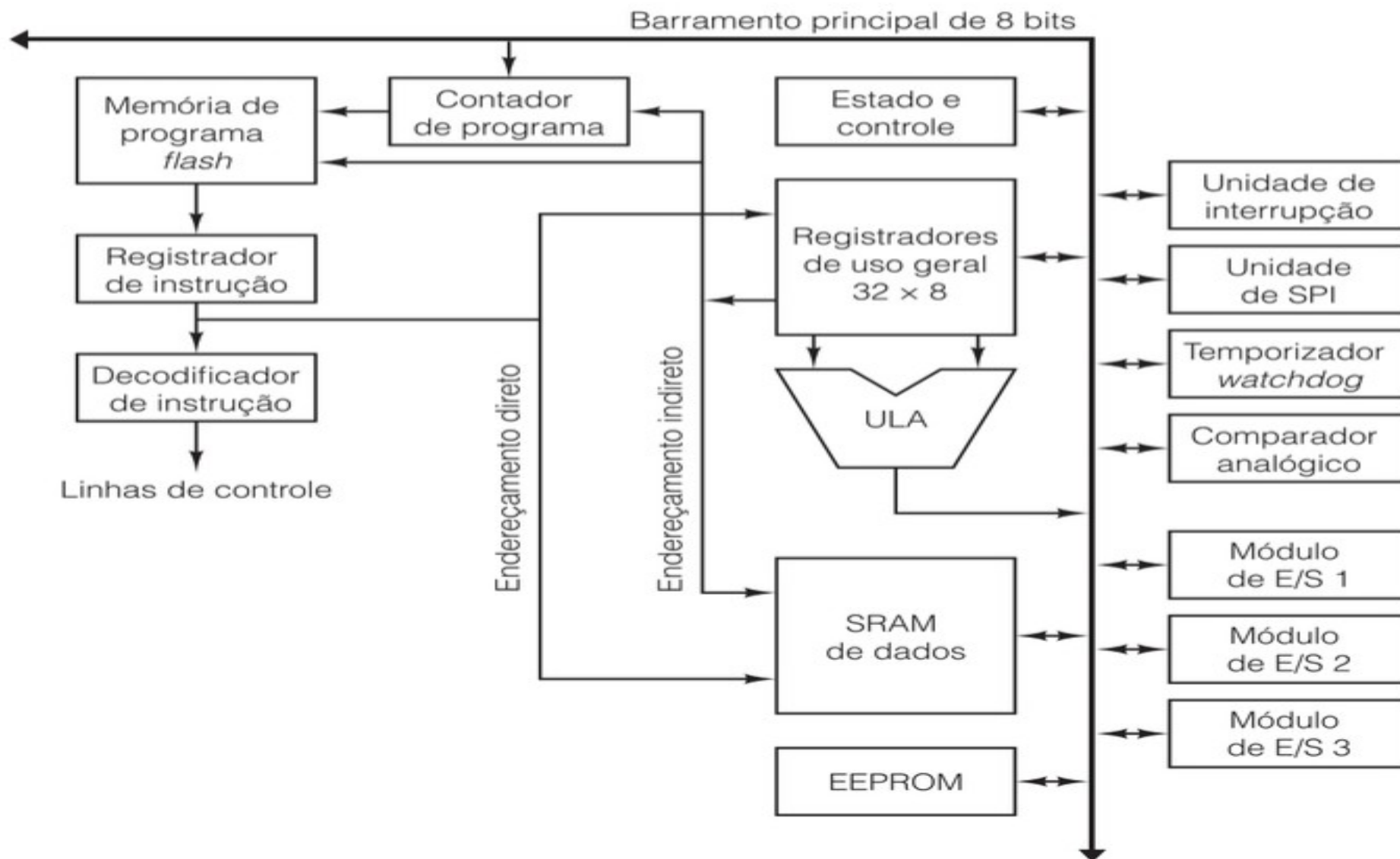
9 Uma representação simplificada do *pipeline* do Cortex A9 da CPU OMAP4430.



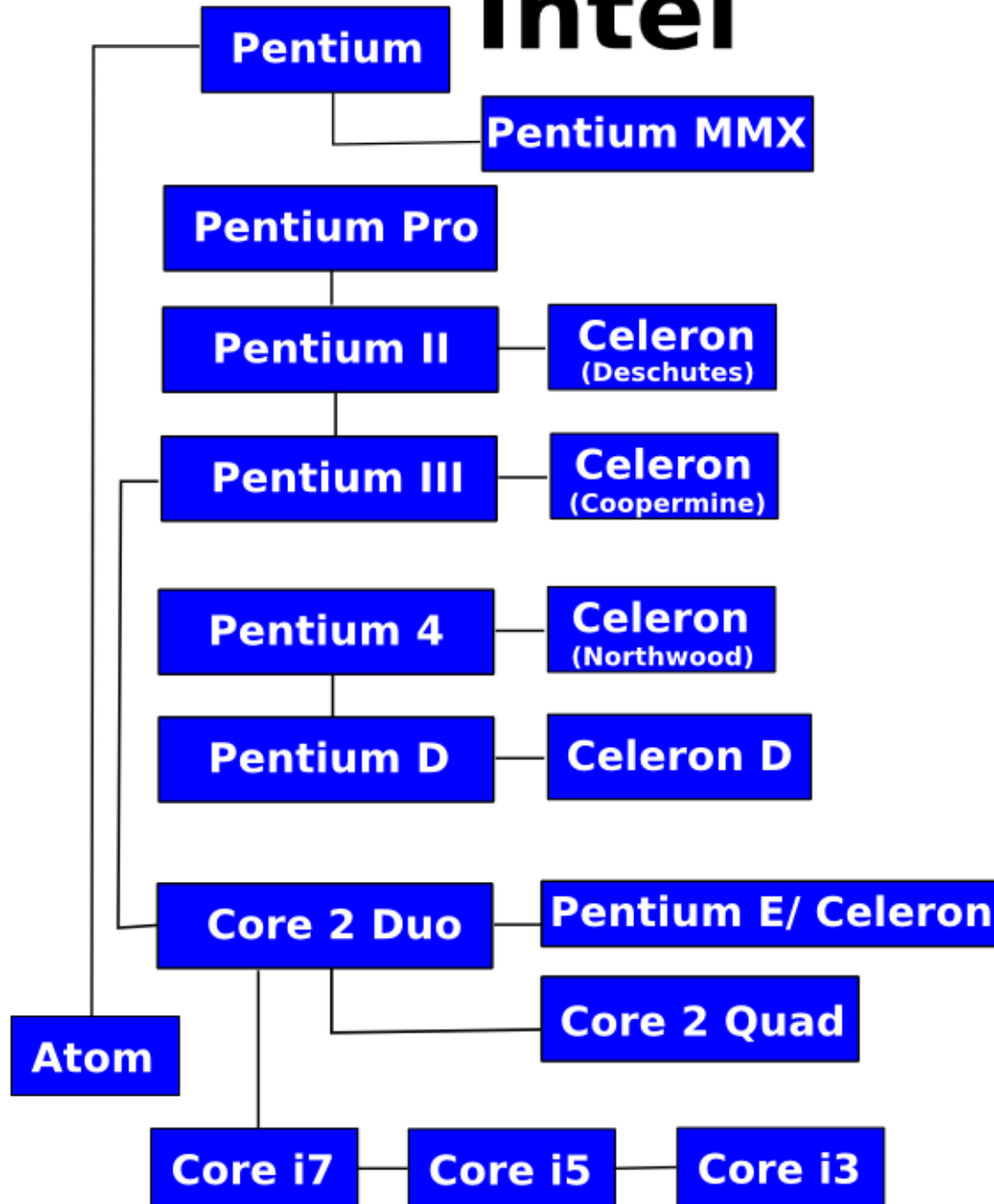
- Unidades funcionais
 - ULA1 de inteiros
 - ULA2 de inteiros
 - Unidade de multiplicação
 - Unidade de ponto flutuante e vetor de SIMD(opcional com suporte VFP e NEON)
 - Unidade de carga e armazenamento (*load/store*)

Microarquitetura do ATmega168

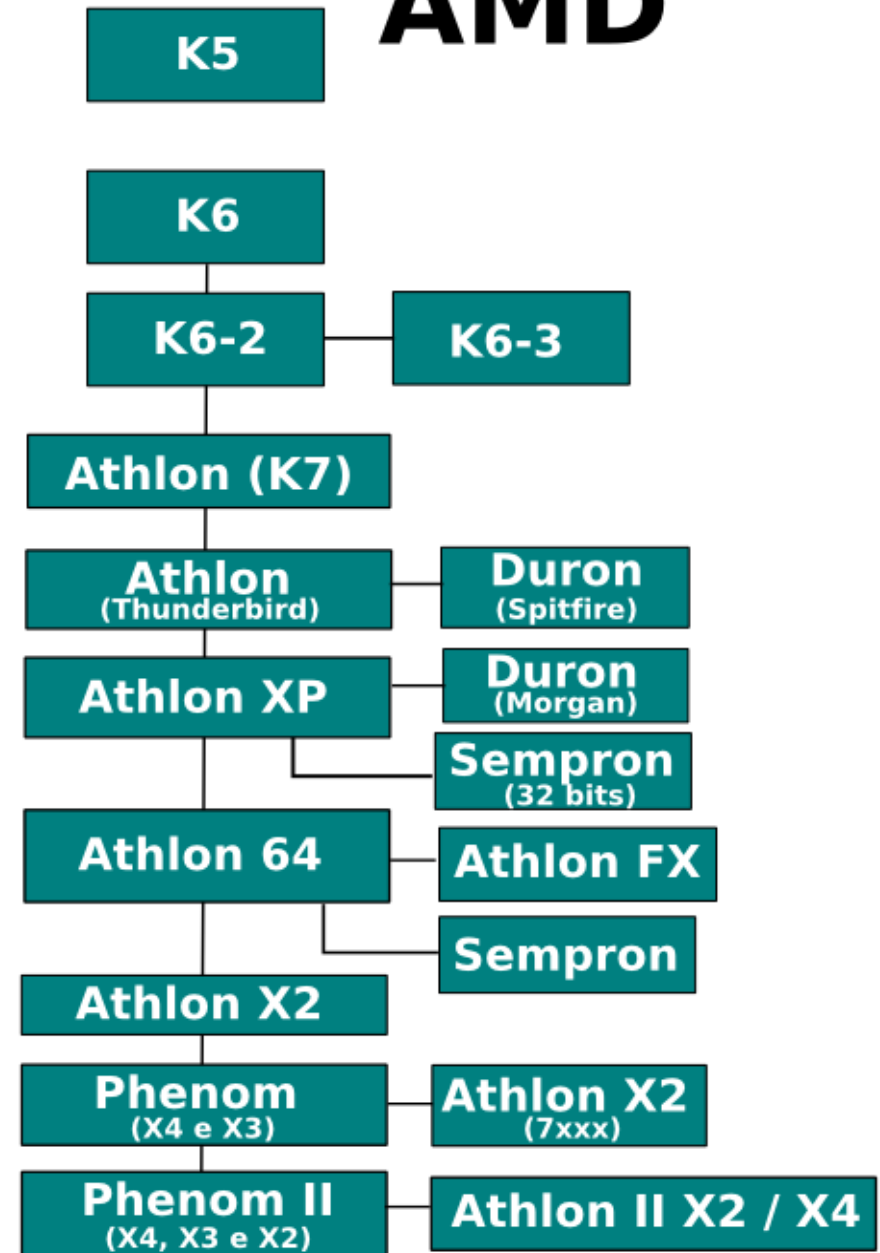
Microarquitetura do ATmega168.



Intel



AMD



Bibliografia

- Organização Estruturada de Computadores – Andrew S. Tanenbaum, Tood Austin – 6ª Edição 2013 Prentice Hall
- Vários sites da internet.